# DOCUMENTAÇÃO TÉCNICA

## SPRINT 4 · COMPUTATIONAL THINKING USING PYTHON

Challenge FIAP 2026 | Zyra: pontos SoulUp e mobilidade urbana


### Integrantes

Carolina Kotz Rodrigues — RM 571876

Felipe Motitsuki Tan — RM 570770

Gabriel Feitosa de Lima — RM 573026

João Gabriel Dantas Moura — RM 573101

Záyon Alves Marques — RM 571205


Documentação técnica do projeto Zyra para a Sprint 4 de Computational Thinking Using Python: Oracle, CRUD, JSON, Open-Meteo, React e melhorias recentes de instalação e execução.

# Sumário

Integrantes ..... 1
Sumário ..... 2
1. Descritivo do projeto, objetivos e justificativa ..... 4
1.1 Objetivo geral ..... 4
1.2 Objetivos específicos ..... 4
1.3 Justificativa e limites da proposta ..... 4
2. Evolução da Sprint 3 para a Sprint 4 ..... 5
3. Arquitetura e organização dos arquivos ..... 6
3.1 Estrutura resumida ..... 6
4. Funcionalidades implementadas e menus ..... 7
4.1 Submenu de gerenciamento de usuários ..... 7
4.2 Submenu SQL e página web ..... 7
5. Banco de dados Oracle e modelagem SQL ..... 8
5.1 Tabela ZYRA_USUARIOS ..... 8
5.2 Tabela ZYRA_TROCAS ..... 8
5.3 Preparação e diagnóstico ..... 8
6. Operações CRUD e regras de negócio ..... 9
6.1 CREATE — Inserção ..... 9
6.2 READ — Consulta ..... 9
6.3 UPDATE — Alteração ..... 9
6.4 DELETE — Exclusão ..... 9
6.5 Regras numéricas e catálogo ..... 9
6.6 Exemplo demonstrável ..... 9
7. Exportação de consultas em JSON ..... 10
7.1 Exemplo simplificado de arquivo ..... 10
8. Consumo da API externa Open-Meteo ..... 11
8.1 Informações processadas ..... 11
8.2 Etapas técnicas da integração ..... 11
9. Integração com o front-end React ..... 12
9.1 Fluxo de comunicação ..... 12
9.2 Endpoints principais ..... 12
10. Estruturas, principais funções e tratamento de erros ..... 13
10.1 Estruturas de programação utilizadas ..... 13

10.2 Funções e métodos representativos ..... 13
10.3 Tratamento de erros e consistência ..... 13
11. Testes, execução e instalação ..... 14
11.1 Pré-requisitos ..... 14
11.2 Passo a passo para o executar ..... 14
11.3 Cenários de teste recomendados ..... 14
12. Uso de inteligência artificial e análise crítica ..... 15
13. Vídeo explicativo e roteiro de apresentação ..... 16
14. Conclusão e referências ..... 17
14.1 Referências técnicas ..... 17

# 1. Descritivo do projeto, objetivos e justificativa

O Zyra é um protótipo de uma plataforma de fidelidade que relaciona missões sustentáveis, acúmulo de pontos SoulUp e benefícios de mobilidade urbana. O usuário cadastrado recebe um saldo inicial e pode converter pontos em passagens simuladas, acompanhar seu histórico e consultar seus resultados.

A solução foi desenvolvida na disciplina Computational Thinking Using Python, da FIAP, para aplicar programação estruturada, validação de dados, modelagem relacional, operações CRUD, consumo de API pública e integração com uma interface web. Ao longo da Sprint 4, o armazenamento principal foi migrado de JSON para Oracle Database.

## 1.1 Objetivo geral

Construir uma aplicação funcional e demonstrável de gerenciamento de usuários, pontos e resgates, com dados persistentes no Oracle, atendimento por terminal e navegador, exportação em JSON e acesso a informações externas relevantes ao deslocamento urbano.

## 1.2 Objetivos específicos

- Implementar inserção, consulta, alteração e exclusão de usuários com SQL.
- Garantir que créditos, descontos e históricos sejam persistidos corretamente.
- Permitir exportar os resultados das consultas para arquivos JSON.
- Consumir dados meteorológicos da Open-Meteo sem exigir cadastro ou chave.
- Compartilhar informações entre interface React e programa Python via API HTTP local.
- Apresentar organização modular, validação de dados e tratamento de erros.

## 1.3 Justificativa e limites da proposta

O transporte público e a sustentabilidade são temas próximos à realidade urbana. O Zyra utiliza uma mecânica de recompensas para tornar a relação entre hábitos sustentáveis e benefícios de mobilidade mais concreta. Na Sprint 4, o resgate é demonstrativo: o sistema não emite bilhetes oficiais nem está integrado à plataforma TOP, à Autopass ou a sistemas reais de pagamento.

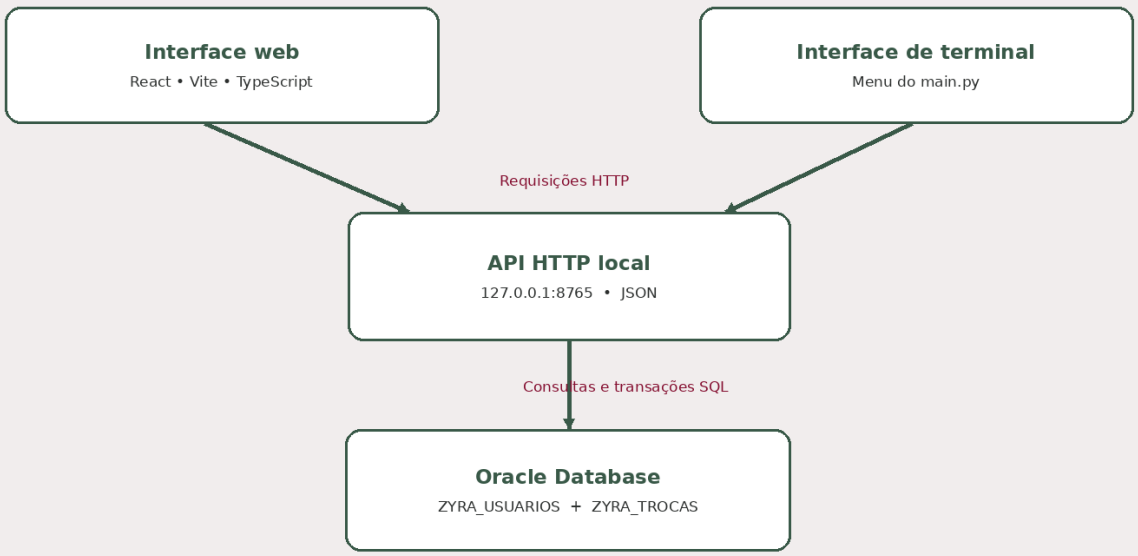
*Os valores de passagem e de pontos são parâmetros do protótipo, preservados das Sprints anteriores; não representam uma consulta automática de tarifas oficiais.*

# 2. Evolução da Sprint 3 para a Sprint 4

| Aspecto | Sprint 3 | Sprint 4 |
|---|---|---|
| Persistência | Arquivo JSON com lista de usuários | Oracle Database como fonte principal |
| CRUD | Operações sobre listas/dicionários | INSERT, SELECT, UPDATE e DELETE em SQL |
| Histórico | Armazenado dentro do usuário | Tabela relacional de trocas |
| Exportação | JSON para guardar dados | JSON gerado a partir de SELECT |
| Integração externa | Não central | Consulta HTTP à Open-Meteo |
| Interface | Terminal Python | Terminal + React via API local |
| Conexão | Não havia banco | Oracle configurável, reconexão opcional e verificação automática de bibliotecas |

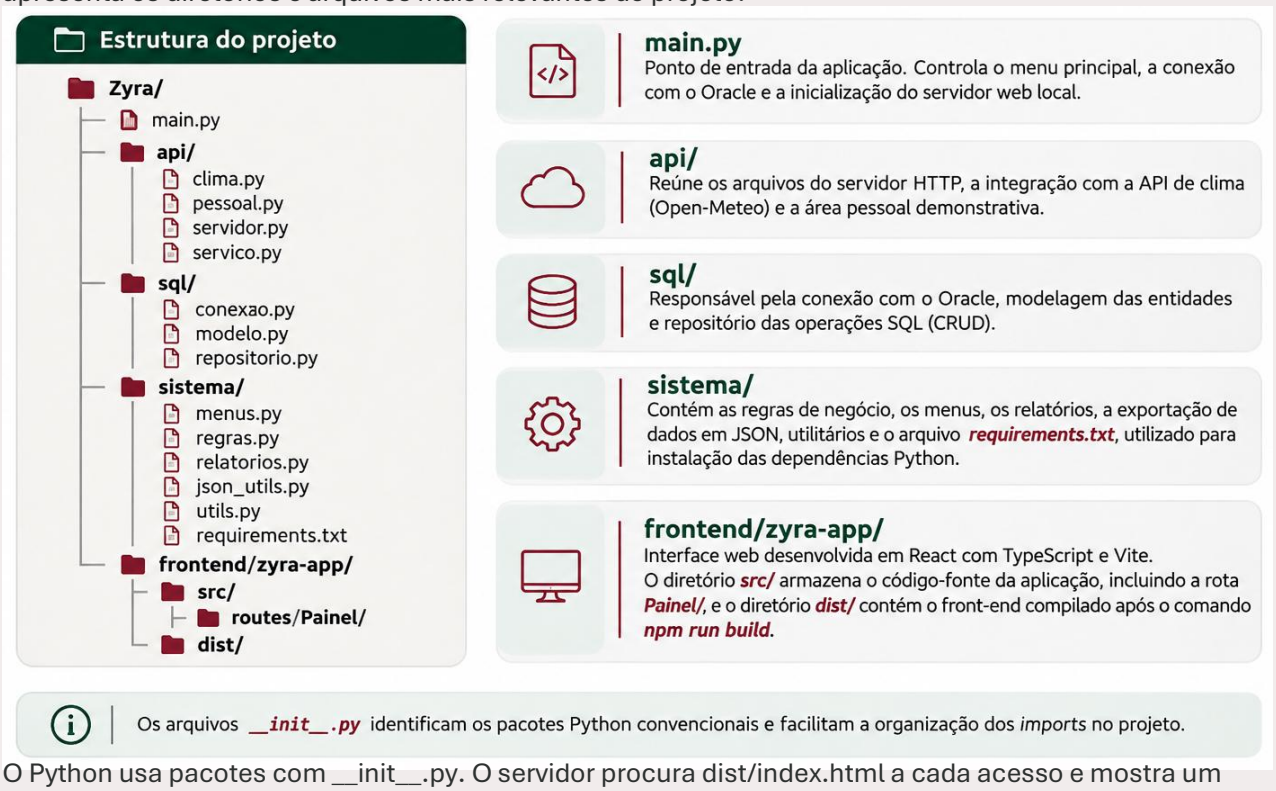
# 3. Arquitetura e organização dos arquivos

O Zyra possui entrada única em main.py. A camada sistema/ organiza menus e regras; sql/ acessa o Oracle; api/ oferece serviços HTTP e clima; frontend/zyra-app/ contém o React. O terminal e a web compartilham os dados do banco.




## 3.1 Estrutura resumida

A aplicação está organizada em quatro áreas principais, além do arquivo de entrada. A estrutura abaixo apresenta os diretórios e arquivos mais relevantes do projeto:




O Python usa pacotes com __init__.py. O servidor procura dist/index.html a cada acesso e mostra um guia de compilação caso ele falte.

# 4. Funcionalidades implementadas e menus

| Menu principal | Funcionalidade |
|---|---|
| [1] Gerenciar usuários | Cadastrar, listar, consultar por ID, alterar nome/saldo e excluir. |
| [2] Pontos e passagens | Creditar pontos, apresentar catálogo e resgatar passagens. |
| [3] Histórico de trocas | Consultar registros associados a um usuário. |
| [4] Resumo do sistema | Apresentar total de usuários, pontos, viagens e valor resgatado. |
| [5] Exportar consultas JSON | Exportar todos, usuário, histórico ou resumo. |
| [6] Condições da viagem | Consultar temperatura e previsão pela Open-Meteo. |
| [7] Conectar SQL | Configurar, salvar, reutilizar conexão e validar/criar tabelas. |
| [8] Abrir página web | Iniciar servidor local com React compilado. |
| [0] Encerrar | Fechar servidor HTTP e conexão Oracle. |

## 4.1 Submenu de gerenciamento de usuários

O submenu reúne CREATE, READ, UPDATE e DELETE. Cada usuário recebe 500 pontos ao cadastrar; o Oracle guarda saldos e históricos. Uma exclusão confirmada remove também as trocas vinculadas pela chave estrangeira.

## 4.2 Submenu SQL e página web

O menu [7] configura SID ou serviço Oracle, diagnostica/cria tabelas e pode salvar credenciais localmente. Na inicialização, o programa tenta reconectar uma configuração salva. A opção [8] inicia a web em 127.0.0.1:8765, com logs HTTP silenciados para não atrapalhar o terminal.

# 5. Banco de dados Oracle e modelagem SQL

O Oracle Database é a persistência principal. A biblioteca oracledb realiza a conexão em modo Thin. A estrutura contém duas tabelas vinculadas por chave estrangeira, com verificações de integridade para nomes, saldos e resgates.

## 5.1 Tabela ZYRA_USUARIOS

| Campo | Tipo | Descrição |
|---|---|---|
| ID | NUMBER(10), IDENTITY, PK | Identificador gerado pelo Oracle. |
| NOME | VARCHAR2(120 CHAR), NOT NULL | Nome validado pelo sistema. |
| PONTOS | NUMBER(12), DEFAULT 500 | Saldo não negativo. |

## 5.2 Tabela ZYRA_TROCAS

| Campo | Tipo | Descrição |
|---|---|---|
| ID | NUMBER(10), IDENTITY, PK | Código da operação. |
| USUARIO_ID | NUMBER(10), FK | Referência ao usuário. |
| PASSAGEM | VARCHAR2(80 CHAR) | Modalidade resgatada. |
| VIAGENS | NUMBER(8) | Quantidade de viagens. |
| CUSTO_PONTOS | NUMBER(12) | Pontos descontados. |
| VALOR_REAIS | NUMBER(12,2) | Valor da conversão. |
| DATA_TROCA | TIMESTAMP | Data/hora da operação. |

O relacionamento é de um usuário para várias trocas (1:N). O índice IX_ZYRA_TROCAS_USUARIO facilita a consulta do histórico por USUARIO_ID. A chave estrangeira possui ON DELETE CASCADE, portanto excluir um usuário exclui suas trocas associadas.

## 5.3 Preparação e diagnóstico

A opção [7] verifica a conexão, as colunas, restrições e o índice, preparando apenas estruturas ausentes. Credenciais salvas ficam em sistema/conexao_oracle.local.json, incluindo a senha em texto simples. A conexão pode ser reutilizada na próxima execução.

# 6. Operações CRUD e regras de negócio

## 6.1 CREATE — Inserção

O cadastro solicita um nome válido e executa INSERT na tabela ZYRA_USUARIOS. O ID é gerado no Oracle e o saldo inicial recebe o valor padrão de 500 pontos. O mesmo recurso é acessível no navegador via POST /api/usuarios.

## 6.2 READ — Consulta

A aplicação oferece SELECT para listar todos os usuários e consultar um cadastro por ID. Quando necessário, o repositório busca também o histórico de resgates; listagens resumidas podem dispensá-lo. No servidor web, os principais endpoints são GET /api/usuarios e GET /api/usuarios/{id}.

## 6.3 UPDATE — Alteração

O operador pode atualizar o nome ou o saldo de pontos, e as operações passam por validação de tipos e limites. Para acrescentar pontos usa-se uma ação específica, separada da alteração absoluta do saldo. A API utiliza PATCH /api/usuarios/{id} e POST /api/usuarios/{id}/pontos.

## 6.4 DELETE — Exclusão

A exclusão é efetuada pelo ID após confirmação no terminal ou no painel React. As trocas relacionadas são removidas pelo relacionamento ON DELETE CASCADE. Na API, o recurso é DELETE /api/usuarios/{id}.

## 6.5 Regras numéricas e catálogo

| Parâmetro | Valor | Significado |
|---|---|---|
| Bônus inicial | 500 pontos | Recebido no cadastro. |
| Passagem unitária | 587 pontos / 1 viagem | Valor simulado de R$ 5,40. |
| Ida e volta | 1.174 pontos / 2 viagens | Duas passagens unitárias. |
| Pacote de 5 | 2.935 pontos / 5 viagens | Cinco passagens unitárias. |
| Saldo máximo | 999.999.999.999 pontos | Limite compatível com NUMBER(12). |

O nome deve conter de 3 a 120 caracteres, com pelo menos três letras, aceitando acentos, espaços, hífens e apóstrofos. Créditos devem ser inteiros positivos; saldos não podem ficar negativos. Um resgate só ocorre com pontos suficientes: o desconto e o registro do histórico são feitos na mesma transação, com rollback quando uma das etapas falha.

## 6.6 Exemplo demonstrável

1. Cadastrar um usuário: recebe 500 pontos.
2. Creditar mais 100: saldo passa para 600.
3. Resgatar uma passagem unitária: descontar 587.
4. Verificar saldo final de 13 pontos e histórico com 1 resgate.
5. Tentar uma segunda passagem: operação rejeitada por saldo insuficiente.

# 7. Exportação de consultas em JSON

O Oracle é a fonte principal; JSON é apenas uma exportação. O módulo sistema/exportacao.py consulta os dados e salva arquivos .json UTF-8 com nomes exclusivos em exportacoes/, preservando acentos e a data de geração.

| Opção | Conteúdo exportado |
|---|---|
| Todos os usuários | Lista de cadastros, total e respectivos históricos. |
| Usuário por ID | Dados detalhados do usuário selecionado. |
| Histórico por usuário | Identificação, quantidade de trocas e registros. |
| Resumo geral | Totais de usuários, saldo, resgates e valores. |

## 7.1 Exemplo simplificado de arquivo

```
{
  "tipo": "usuario",
  "usuario": {
    "id": 1,
    "nome": "Usuário Exemplo",
    "pontos": 13,
    "historico": []
  },
  "exportado_em": "2026-11-03T14:00:00-03:00"
}
```

Exemplo ilustrativo de estrutura, não extração de dados reais. As funções verificam usuário inexistente, erro de escrita e possibilidade de colisão de nomes; arquivos anteriores não são sobrescritos.

# 8. Consumo da API externa Open-Meteo

O Zyra consome a API pública Open-Meteo por meio de requisição HTTP GET, usando requests. A URL consultada é https://api.open-meteo.com/v1/forecast, com coordenadas da cidade de São Paulo, fuso America/Sao_Paulo e horizonte de três dias. A integração não modifica o banco Oracle e não exige chave de API para o uso demonstrativo previsto.

## 8.1 Informações processadas

- Condições atuais: temperatura, sensação térmica, umidade, precipitação, vento e código meteorológico.
- Previsão diária: temperaturas mínima e máxima, chance de chuva, precipitação e descrição textual.
- Conversão dos códigos meteorológicos em descrições compreensíveis, como Céu limpo ou Encoberto.
- Cache de aproximadamente cinco minutos para reduzir requisições repetidas.

## 8.2 Etapas técnicas da integração

6. Montar os parâmetros da requisição HTTP com localização, campos e fuso horário.
7. Enviar GET ao serviço da Open-Meteo e receber uma resposta JSON.
8. Validar os campos numéricos, datas, listas e valores opcionais.
9. Converter códigos meteorológicos em mensagens em português.
10. Exibir o resultado no terminal e disponibilizar GET /api/clima ao React.

Falhas de conexão, timeout, limites de requisição, respostas HTTP inadequadas ou JSON inesperado são tratados por erros específicos. A aplicação pode continuar executando as outras funcionalidades mesmo quando a API externa está indisponível.

# 9. Integração com o front-end React

O front-end utiliza React, TypeScript, Vite e React Router. A rota /painel permite CRUD, pontos, passagens, histórico, resumo, exportação JSON e consulta climática. api/pessoal.py oferece rotas demonstrativas de identificação por nome ou ID, sem autenticação segura por senha.

## 9.1 Fluxo de comunicação

11. O usuário realiza uma operação na rota /painel.
12. O cliente src/services/zyraApi.ts envia HTTP com corpo JSON quando necessário.
13. O servidor api/servidor.py recebe a requisição em 127.0.0.1:8765.
14. O serviço api/servico.py valida a rota, chama sql/repositorio.py e devolve JSON.
15. O React atualiza a interface de acordo com o estado gravado no Oracle.

## 9.2 Endpoints principais

| Método | Rota | Finalidade |
|---|---|---|
| GET | /api/status | Estado do serviço e conexão SQL. |
| GET | /api/passagens | Catálogo de passagens. |
| GET / POST | /api/usuarios | Listar ou cadastrar usuários. |
| GET / PATCH / DELETE | /api/usuarios/{id} | Consultar, alterar ou excluir. |
| POST | /api/usuarios/{id}/pontos | Creditar pontos. |
| POST | /api/usuarios/{id}/resgates | Resgatar passagem. |
| GET | /api/resumo | Indicadores gerais. |
| GET | /api/clima | Condições meteorológicas. |

No desenvolvimento, o Vite encaminha /api ao Python local. O main.py serve o React compilado em frontend/zyra-app/dist. Sem o build, api/servidor.py exibe instruções para executar npm ci e npm run build. O servidor omite os logs de acesso HTTP para manter o menu legível.

# 10. Estruturas, principais funções e tratamento de erros

## 10.1 Estruturas de programação utilizadas

| Recurso | Aplicação no Zyra |
|---|---|
| Constantes e tuplas | Bônus de 500 pontos, catálogo fixo de passagens e limites. |
| Dicionários e listas | Objetos JSON, dados de usuários, históricos e respostas da API. |
| Funções e classes | Separação entre validações, repositório Oracle e serviços HTTP. |
| if / else e match-case | Decisões de saldo, menus e operações selecionadas. |
| while e for | Menu contínuo, validação de entradas e leitura de coleções. |
| try / except / finally | Captura de falhas, rollback e liberação de conexões. |
| Context managers | Controle do ciclo de vida da conexão em requisições HTTP. |

## 10.2 Funções e métodos representativos

| Arquivo / método | Responsabilidade |
|---|---|
| main.py — verificar_dependencias / Zyra.executar | Instala pacotes ausentes e controla o menu principal. |
| main.py — Zyra.conectar_sql | Gerencia credenciais, criação e diagnóstico de tabelas. |
| sql/conexao.py — conectar | Abre sessão Oracle a partir dos parâmetros informados. |
| sql/repositorio.py — criar_usuario | Executa INSERT e recupera o identificador. |
| sql/repositorio.py — resgatar_passagem | Desconta pontos e registra troca em transação. |
| sistema/exportacao.py — exportar_consulta | Gera arquivos JSON de consultas SQL. |
| api/servico.py — ServicoZyra.processar | Despacha requisições HTTP da interface web. |
| api/servidor.py — Roteador / log_request | Serve React e API; evita logs HTTP no terminal. |

## 10.3 Tratamento de erros e consistência

O sistema valida nomes, IDs, créditos e saldos antes de alterar registros. O repositório utiliza parâmetros SQL e transações para manter o débito de pontos e a troca consistentes. A função pausar() preserva avisos importantes até o usuário pressionar Enter.

A API utiliza códigos HTTP como 400, 401, 404, 409, 502 e 503. Há tratamento de exceções e rollback no Oracle. A área pessoal possui sessão temporária, mas não comprova identidade; o acesso é demonstrativo e restrito ao computador local.

# 11. Testes, execução e instalação

## 11.1 Pré-requisitos

- Python compatível com o projeto e dependências do arquivo sistema/requirements.txt.
- Oracle Database acessível, com credenciais próprias e permissão para criar tabelas e índice no esquema do avaliador.
- Para recompilar o React: Node.js/npm compatíveis com as versões de React Router e Vite do projeto.
- Acesso à internet apenas para dependências, Oracle remoto e Open-Meteo (quando necessários).

## 11.2 Passo a passo para o executar

1. Extrair o ZIP em uma pasta local.
2. Executar **python main.py** dentro do terminal (pode ser pelo terminal do sistema ou dentro do VS Code). Se faltar requests ou oracledb, o programa tenta instalá-los automaticamente via pip; em caso de falha, usar python -m pip install -r sistema/requirements.txt.
3. No menu principal, selecionar [7] Conectar SQL / preparar tabelas.
4. Informar credenciais próprias, servidor, porta e SID ou nome do serviço Oracle.
5. Autorizar a criação ou verificação das tabelas e conferir o diagnóstico.
6. Testar cadastro, consulta, alteração, exclusão, créditos, resgates, histórico, exportação JSON e clima.
7. Executar **cd frontend/zyra-app**, **npm ci** e por fim **npm run build** (comandos em sequência necessários para que a página web funcione corretamente).
8. Retornar à raiz, selecionar [8] Abrir página web e acessar **http://127.0.0.1:8765/painel**. A página provisória explica como compilar se ainda faltar o build.

## 11.3 Cenários de teste recomendados

| Cenário | Resultado esperado |
|---|---|
| Cadastrar / consultar | Novo usuário com ID próprio e 500 pontos. |
| Editar nome e pontos | Atualização persistente no Oracle. |
| Crédito + resgate | 500 + 100 − 587 = 13 pontos; 1 troca. |
| Saldo insuficiente | Operação rejeitada, sem descontar pontos. |
| Exportar todos / histórico | Arquivo .json válido, consultado do Oracle. |
| Consultar API | JSON de clima exibido ou erro tratado sem travar. |
| React compilado | Após npm ci/build, mesma base Oracle no terminal e no painel. |
| Sem Oracle / sem build | Exibe avisos e guia; operações SQL ficam indisponíveis. |

# 12. Uso de inteligência artificial e análise crítica

Durante o projeto, usamos ChatGPT para esclarecer dúvidas, propor organização de módulos, revisar erros e apoiar a integração Python, Oracle e React. As sugestões exigiram verificação pela equipe.

Entre as solicitações, destacam-se a solicitação para fatorar o código em um arquivo de execução principal main.py e pastas para implementar as demais funcionalidades e a verificação de erros após desenvolver alguma função.

Analisamos o código, testamos fluxos e ajustamos sugestões que não atendiam ao contexto. Estudamos na prática condicionais, funções, listas, dicionários, exceções, JSON, HTTP e SQL; o uso de IA não substituiu a avaliação dos resultados.

# 13. Vídeo explicativo e roteiro de apresentação

https://www.youtube.com/watch?v=BV1OGFuA_Ic

# 14. Conclusão e referências

A Sprint 4 consolidou o Zyra como uma aplicação integrada e funcional para o contexto acadêmico. A migração do armazenamento em JSON para Oracle, a implementação do CRUD SQL, o registro transacional de resgates, a exportação de consultas, o consumo de uma API externa e o painel React demonstram a aplicação prática dos conteúdos da disciplina.

O projeto também evidenciou a importância de testar incrementos, separar responsabilidades, documentar regras de negócio e revisar erros encontrados na execução real.

## 14.1 Referências técnicas


- OPEN-METEO. Weather Forecast API — documentação oficial. https://open-meteo.com/en/docs
- PYTHON SOFTWARE FOUNDATION. Python Documentation. https://docs.python.org/3/
- ORACLE. python-oracledb documentation. https://python-oracledb.readthedocs.io/
- ORACLE. Oracle Database SQL Language Reference. https://docs.oracle.com/en/database/
- META. React Documentation. https://react.dev/
- VITE. Vite Documentation. https://vite.dev/guide/
- FIAP. Enunciado da Sprint 4 — Computational Thinking Using Python (material fornecido à equipe).


**ZYRA • SPRINT 4 • FIAP 2026**